

Report - Version 2: Neural Character Model with NumPy

Introduction

Version 2 extends the character language model by replacing direct transition counting with a small neural model implemented in NumPy.

The corpus, vocabulary, adjacent-character task, and autoregressive generation loop remain unchanged. This isolates the new idea: probabilities are now produced by trainable parameters.

```
Characters
↓
Integer identifiers
↓
One-hot inputs
↓
Trainable weights
↓
Softmax probabilities
↓
Generated text
```

Goal of Version 2

The model should be able to:

- reuse the corpus and character pairs from the previous version;
- convert characters into integer identifiers and one-hot vectors;
- represent predictions with a trainable weight matrix;
- convert logits into probabilities with softmax;
- measure prediction error with cross-entropy;
- learn parameters through gradient descent;
- generate reproducible text from the learned probabilities.

The version remains deliberately small so that every calculation can be inspected directly.

Architecture

The complete Version 2 flow is:

```
Corpus
↓
Vocabulary and character IDs
↓
Adjacent input-target pairs
↓
One-hot input matrix
↓
Trainable weight matrix
↓
Logits and softmax
↓
Cross-entropy and gradient descent
↓
Autoregressive generator
```

The context still contains one character. The newly sampled character becomes the context for the next generation step.

Numerical representation

Neural calculations operate on numbers. Each of the 27 characters in the vocabulary receives an integer identifier.

```
model
↓
[15, 17, 7, 8, 14]

m -> o becomes 15 -> 17
o -> d becomes 17 -> 7
d -> e becomes 7 -> 8
e -> l becomes 8 -> 14
```

The corpus contains 440 characters and produces 439 adjacent input-target examples.

One-hot inputs

Every input ID selects a one-hot vector of length 27: one position is equal to 1 and all remaining positions are 0.

```
Input matrix shape: (439, 27)
```

Trainable weight matrix

The model contains one 27 by 27 weight matrix. Each row corresponds to a current character and each column to a possible next character.

```
Weight matrix shape: (27, 27)
Trainable parameters: 729
```

Multiplying a one-hot input by the matrix selects the row associated with the current character. The 27 resulting values are logits.

The initial weights are small random values generated with a local NumPy seed of 42. This makes the complete run reproducible.

Softmax probabilities

Logits are unrestricted scores. Softmax converts them into a probability distribution over the complete vocabulary.

```
shifted = logits - max(logits)
probabilities = exp(shifted) / sum(exp(shifted))
```

Subtracting the maximum logit improves numerical stability without changing the resulting probabilities.

- every probability is between 0 and 1;
 - the 27 probabilities for one context sum to 1;
 - higher logits produce higher probabilities.
-

What changed

The statistical model calculated probabilities directly from observed counts. The neural model produces them from parameters that must be learned.

Cross-entropy loss

Cross-entropy measures how much probability the model assigns to the correct next character.

```
loss = -mean(log(probability of the correct target))
```

A high probability for the correct target produces a low loss. A low probability produces a high loss.

Gradient descent

Every training step performs four operations:

- calculate logits and probabilities for all 439 examples;
- measure the average cross-entropy loss;
- calculate the gradients of logits and weights;
- update the weights in the direction that reduces the loss.

```
logits gradient = probabilities - targets  
weights gradient = inputs.T @ logits gradient  
weights -= learning rate * weights gradient
```

Training uses full-batch gradient descent with learning rate 20.0 and 1,000 updates. The calculations are implemented directly in NumPy, without automatic differentiation.

Training result

```
Initial loss: 3.297271  
Final loss: 1.946110
```

The steady reduction in loss shows that the matrix has learned the character transitions in the corpus.

Learned probabilities

After training, one row of the weight matrix can be inspected as the next-character distribution for a chosen context.

For the character `m`, the highest probabilities are:

```
a: 0.2212 e: 0.2212 o: 0.2212
space: 0.1101 p: 0.1101 s: 0.1101
```

These values closely reproduce the empirical transition distribution. Softmax also assigns a very small non-zero probability to transitions that were not observed.

Neural text generation

Generation preserves the same autoregressive loop used previously:

- convert the current character into its integer ID;
- select its trained logits and apply softmax;
- sample the next ID from the probability distribution;
- convert the sampled ID back into a character;
- use that character as the next context.

A local NumPy seed makes the 300-character demonstration reproducible.

Included checks

The notebook assertions verify:

- the number and shape of the numerical examples;
- valid one-hot vectors and parameter dimensions;
- a final loss lower than the initial loss;
- next-character probabilities sum to 1;
- reproducible generation and the requested output length.

A complete run ends with the message `All checks passed.`

What Version 2 teaches

The central progression is:

encode -> predict -> measure -> differentiate -> update -> sample

Version 2 shows how a probability distribution can be learned through optimization instead of being calculated directly from counts.

Interpretation

The model is neural because its predictions come from trainable parameters optimized with gradient descent. It is still a character bigram model because only one preceding character is used as context.

Its expressive capacity remains similar to the statistical model. The advance is parameter learning, not yet better long-range quality.

Limitations

- the context contains one character only;
 - the corpus is small and embedded in the notebook;
 - training and inspection use the same examples;
 - all examples are processed together in one full batch;
 - there is no hidden layer, embedding, automatic differentiation, or framework;
 - generated text has local patterns but no long-term coherence.
-

Conclusion

Version 2 is the first trainable neural model in Building LLM. It turns characters into numerical inputs, learns a weight matrix with NumPy, and generates text from the resulting probability distributions.